

DELIVERABLE 3

Fielding Workflow Design: Surveyor Python Pipeline

Goal. Build a Python workflow that takes the finalized Utah AI data center questionnaire and polling plan, converts them into Surveyor-ready survey YAML, validates the instrument, launches fielding, monitors completes and quotas, closes fielding, and exports topline/crosstabs. The Surveyor API supports survey creation, YAML conversion/validation, deployment, fielding through providers, field-status monitoring, exports, and reports.

Workflow diagram.

Questionnaire + polling plan

- Convert survey draft to YAML
- Validate YAML
- Create survey
- Review survey + deploy
- Field through provider
- Monitor completes / quotas / LOI
- Pause or update fielding if needed
- Close fielding
- Export CSV / generate reports

API sequence.

1. **Preflight checks.** Call `/v1/info/health` and `/v1/info/version` to confirm the API is reachable and compatible.
2. **Convert questionnaire to YAML.** Send the final text questionnaire to `/v1/surveys/survey_to_yaml`. Store the returned YAML locally as a versioned artifact.
3. **Validate survey design.** Submit YAML to `/v1/surveys/validate`. If validation fails, stop the workflow and write errors to a launch checklist.
4. **Create survey.** Use POST `/v1/surveys` with title and finalized question elements. Store the returned `survey_id`.
5. **Review deployment status.** Use GET `/v1/surveys/{survey_id}` and GET `/v1/surveys/{survey_id}/deploy_status` to confirm the programmed survey matches the questionnaire.
6. **Deploy.** Use POST `/v1/surveys/{survey_id}/deploy` to make the survey available.
7. **Field survey.** Use POST `/v1/surveys/{survey_id}/field` with provider, requested completes, quotas, and targeting from the polling plan. Surveyor supports fielding through providers such as Cint marketplace and Research Desk.
8. **Monitor launch.** Use GET `/v1/surveys/{survey_id}/field/{field_id}/status` for completion percentage, completes per hour, and estimated completion time. Use the completion-time endpoint to monitor LOI percentiles and catch speeders or unexpected respondent behavior.
9. **Adjust fielding if needed.** Use PATCH `/v1/surveys/{survey_id}/field/{field_id}` to adjust fielding parameters if quotas are soft-failing or completes are coming in unevenly.

10. **Close fielding.** When completes and quotas are met, use POST /v1/surveys/{survey_id}/undeploy to stop new responses.

11. **Export and report.** Export complete responses using /v1/surveys/{survey_id}/export/csv and generate reports using /v1/surveys/{survey_id}/reports.

Pseudocode.

```
Python
client = SurveyorClient(api_key=API_KEY)

client.health_check()

yaml = client.convert_to_yaml(questionnaire_text)
validation = client.validate_yaml(yaml)

if not validation.ok:
    raise LaunchBlocker(validation.errors)

survey_id = client.create_survey(
    title="Utah AI Data Centers Survey",
    yaml=yaml
)

client.deploy_survey(survey_id)

field_id = client.field_survey(
    survey_id=survey_id,
    provider="research_desk",
    completes=800,
    quotas=polling_plan.quotas,
    targeting=polling_plan.targeting
)

while True:
    status = client.get_field_status(survey_id, field_id)
    loi = client.get_completion_time(survey_id, field_id)

    log_status(status, loi)

    if status.complete_count >= 800 and
quotas_are_healthy(status):
        break

    if status.quota_problem:
```

```
        client.update_fielding(survey_id, field_id,
revised_parameters)

client.undeploy_survey(survey_id)

csv = client.export_csv(survey_id, completes_only=True,
include_weights=True)
report = client.create_report(survey_id)
```